MBFF Clustering Engine in OpenROAD

Your Repo URL
> https://github.com/ABKCourses/e260c-sp26-mp1-fakl/tree/main

Concept & Implementation

Our initial vision was a greedy clustering approach based on physical distance. In our initial attempt at implementation, an adjacency list was created, with each FF's "neighbors" determined based on whether their distance to the FF was below a threshold. A dirty implementation detail was to do multiple repeated passes, where the distance threshold was gradually loosened to expand clusters (as long as the cluster expansion was substantial). After this, the adjacency list was processed so that as many subclusters of size 4 (greedy) and then subclusters of size 2 could be selected from the clusters. After that, depending on the size of the subcluster (2 or 4), the SBFFs were replaced with appropriate 2 bit or 4 bit MBFFS.

Ultimately, the QoR wasn't the best after evaluation. We realized we needed to move away from this approach because it solely considered physical distance and did not consider timing or power. [Note: old clustering script still available as "naive_clustering.py"]

Another approach we tried was using k-means clustering based on, "Scalable Flip-Flop Clustering Using Divide and Conquer For Capacitated K-Means", however, we ultimately failed to get a good implementation working (we had nonsensical QoR), so we moved on.

The final clustering approach that was settled upon was using physical distance based clustering (our original vision) augmented with heuristics based on power and timing to improve our clustering, based on the approach described in the paper, "Revisit MBFF: Efficient Early-Stage Multi-bit Flip-Flops Clustering with Physical and Timing Awareness." Initially, candidate pairs of flip flops are chosen by measuring Manhattan distance between every possible FF pair and checking if the distance falls below a primary threshold (note: there are expected to be conflicts, where a FF could appear in

multiple candidate pairs). Using these candidate pairs, a conflict graph of all the candidate pairs is created, with each graph's node representing a candidate pair, and the graph's edges connecting nodes with common flip flops. Then, in a greedy approach, we pluck out the most "promising" FF pairs from the candidates: the nodes with the highest "***c***" value are selected, and every time a node is selected, all neighboring nodes which have a common flip flop with the selected node are deleted from the graph along with the selected node itself. The ***c*** value is a heuristic that combines two components: power and timing; refer to the paper [3] (pg. 1233) for the exact formulation, which we have replicated in the Python script. Nodes are greedily selected (based on ***c***) to form the "promising" FF pairs until the conflict graph is emptied. We then group some of the promising FF pairs into 4-tuples. To select the most viable clusters of size 4, we again use a different (secondary) distance threshold, where close pairs of promising FFs are grouped together to form clusters of size four for the 4 bit MBFFs. After all possible 4 bit MBFF clusters are selected, all the leftover promising FF pairs are left as clusters of size 2; finally, the size 2 and size 4 clusters have their SBFFs replaced with 2 bit MBFFs and 4 bit MBFFS.

Source Tour

Using short sentences, bullet points, code snippets, and/or code links describe the flow of your implemented code.

Helper function to retrieve iterm from inst using string name as a input

```
def iterm_by_name(inst, name):
   for iterm in inst.getITerms():
       if name in iterm.getName():
           return iterm
   return None
```

Instances(Insts) from the block are retrieved and filtered to select only flip flops.

```
insts = block.getInsts()
filtered_insts = [i for i in insts if "sg13g2_dfrbpq_1" in i.getMaster().getName() or
"sg13g2_dfrbpq_2" in i.getMaster().getName() ]
```

This loop chooses all the candidate pairs by comparing each instance's Manhattan distance to other flipflops and checking if it falls below the threshold. The threshold is loosened in three passes to find more viable candidate pairs. If the Manhattan distance is below threshold *and*

the FFs in the pair are in the same clock/reset domain (this is what the helper function “compatible” checks), the candidate pair is added to the total list of candidate pairs.

```
changelist = {}
threshold = args.thresh
pass_count = 3
cand_pairs = []
for i in range(pass_count):
   improvement_count = 0
   for j in range(len(filtered_insts)):
       instance = filtered_insts[j]
       if instance not in adjacency_list:
               adjacency_list[instance] = [instance]
       for inst in filtered_insts:
           if(inst!=instance):
               #manhattan distance
               dist = abs(inst.getLocation()[0] - instance.getLocation()[0]) +
abs(inst.getLocation()[1] - instance.getLocation()[1])

               if (dist < threshold and compatible(instance, inst)):
                   if inst not in  adjacency_list[instance]:
                       adjacency_list[instance].append(inst)
                       cand_pairs.append([instance, inst, dist, float('inf')])
                       improvement_count += 1

   threshold += 500
   if (improvement_count < args.improv):
break
   print("improvement count", improvement_count, "threshold", threshold, "pass", i)
```

The masters for 2 bit MBFF and 4 bit MBFF are selected from the library using string searches.

```
MBFF2_master = None
MBFF4_master = None
for lib in design.getDb().getLibs():
   for master in lib.getMasters():
       name = master.getName()
       if "V2X" in name or "V4X" in name or "H2V2X" in name:
           pins = [mterm.getName() for mterm in master.getMTerms()]
       if "V2X_1" in name:
           MBFF2_master = master
       if "V4X_1" in name:
           MBFF4_master = master
```

This section creates the conflict graph by creating a dictionary (adjacency list) with the key (node) being the index of a candidate pair and the (edges) values of neighboring nodes which are candidate pairs which share a common flip flop with the key candidate pair.

```
#create conflict graph
conflict_graph = defaultdict(set)
for i in range(len(cand_pairs)):
```

```
    d1 = cand_pairs[i][2]
    a_1 = cand_pairs[i][0]
    b_1 = cand_pairs[i][1]
    pair_1 = {a_1,b_1}

    for j in range(i+1,len(cand_pairs)):
        d1 = cand_pairs[j][2]
        a_2= cand_pairs[j][0]
        b_2 = cand_pairs[j][1]
        pair_2 = {a_2,b_2}
        if ( pair_1 & pair_2):
            conflict_graph[i].add(j)
            conflict_graph[j].add(i)
```

This a helper function to calculate the slack term (used for the ***c*** heuristic) for each flip flop. The input(D)and output(Q) iterms of each flip flop in the candidate pair are extracted. The timing heuristic then relies on slack values of the input and output iterms.

```
def get_slack_term(inst_i, inst_j):
    d_j = iterm_by_name(inst_j,"/D")
    q_j = iterm_by_name(inst_j,"/Q")
    d_i = iterm_by_name(inst_i,"/D")
    q_i = iterm_by_name(inst_i,"/Q")

    slack_d_j = timing.getPinSlack(d_j, openroad.Timing.Max)
    slack_d_i = timing.getPinSlack(d_i, openroad.Timing.Max)
    slack_q_j = timing.getPinSlack(q_j, openroad.Timing.Max)
    slack_q_i = timing.getPinSlack(q_i, openroad.Timing.Max)

    slack_f_i = slack_d_i - slack_q_i
    slack_f_j = slack_d_j - slack_q_j

    return (slack_f_i  - slack_f_j)/(slack_f_i + slack_f_j) + 1
```

These lines of code calculate the power savings(w) of using a MBFF over two single SBFFs.

```
inst_MBFF = odb.dbInst_create(block, MBFF2_master, "mbff2_master")
power_term  = 2*timing.dynamicPower(instx, corner) -
timing.dynamicPower(inst_MBFF , corner)
```

This loop greedily selects nodes in the conflict graph with highest ***c*** value which is a heuristic which combines the power savings of adjacent nodes and the timing heuristic based on input/output slack (see paper [3] pg 1233 for the formulation). In each iteration, the best node is appended to selected pairs and is deleted along with its neighbours from the conflict graph. The heuristic value is recalculated in each iteration as the number of neighbors may decrease as the loop proceeds.

```
remaining = set(range(len(cand_pairs)))
selected_pairs = []
while (len(remaining)>0):
     for idx in remaining:
       cand_pairs[idx][3] = power_term*len(conflict_graph[idx])/power_term +
get_slack_term(cand_pairs[idx][0], cand_pairs[idx][1])

   best = min(remaining,key=lambda idx: cand_pairs[idx][3])
   selected_pairs.append(tuple(cand_pairs[best][0:2]))
   remaining.remove(best)
   remaining -= conflict_graph[best]
```

This loop merges clusters of size 2 to create clusters of size 4 if their Manhattan distance is below the threshold.

```
four_bit_pairs = []
used = set()
threshold = args.thresh_4_bit
for i in range(len(selected_pairs)):
   best_dist = float('inf')
   best = None
   c1 = selected_pairs[i]
   if c1 in used:
       continue
   for j in range(i+1, len(selected_pairs)):
       c2 = selected_pairs[j]
       if c2 in used:
           continue
       x1 = sum(ff.getLocation()[0] for ff in c1)/2
       y1 = sum(ff.getLocation()[0] for ff in c1)/2
       x2 = sum(ff.getLocation()[0] for ff in c2)/2
       y2 = sum(ff.getLocation()[0] for ff in c2)/2

       dist = abs(x2-x1) + abs(y2-y1)
       if (dist < best_dist):
           best = c2
           best_dist = dist

   if best and best_dist <= threshold:
       four_bit_pairs.append(c1+best)
       used.add(c1)
       used.add(best)
```

```
two_bit_clusters = [
   c for c in selected_pairs
   if c not in used
]
```

Finally, the remaining code traverses through the list of clusters of size 2 and clusters of size 4 and replaces the SBFFs in them with 2 bit MBFFs and 4 bit MBFFS respectively. The appropriate MBFFs instance is created and connected to the nets are extracted from the SBFFs. The location is set to the centroid location of the replaced SBFFs and the status is set to placed.

```
j = 0
for pair in two_bit_clusters:
   j+=1
   name =  "mbff_" + str(j)
   changelist[name] = [z.getName() for z in pair]
   print( "mbff_" + str(j))
   n=2
   inst = odb.dbInst_create(block, mbbf2_master, name)
   x_sum = 0
   y_sum = 0
   for i in pair:
       x_sum += i.getLocation()[0]
       y_sum += i.getLocation()[1]

   x = int(x_sum/n)
   y = int(y_sum/n)
   inst.setLocation(x, y)
   inst.setPlacementStatus("PLACED")

   d0 = iterm_by_name(pair[0],"/D").getNet()
   q0 = iterm_by_name(pair[0],"/Q").getNet()
   d1 = iterm_by_name(pair[1],"/D").getNet()
   q1 = iterm_by_name(pair[1],"/Q").getNet()
   clk_net = iterm_by_name(pair[0],"/CLK").getNet()
   reset_net = iterm_by_name(pair[0],"/RESET_B").getNet()

   iterm_by_name(inst,"/D0").connect(d0)
   iterm_by_name(inst,"/D1").connect(d1)
   iterm_by_name(inst,"/Q0").connect(q0)
   iterm_by_name(inst,"/Q1").connect(q1)
   iterm_by_name(inst,"/CLK").connect(clk_net)
   iterm_by_name(inst,"/RESET").connect(reset_net)

   for i in pair:
       for it in list(i.getITerms()):
```

```
            it.disconnect()
        odb.dbInst_destroy(i)

for pair in four_bit_pairs:
    j+=1
    print( "mbff_" + str(j))
    name =  "mbff_" + str(j)
    changelist[name] = [z.getName() for z in pair]
    n=4
    inst = odb.dbInst_create(block,mbbf4_master, name)
    x_sum = 0
    y_sum = 0
    for i in pair:
        x_sum += i.getLocation()[0]
        y_sum += i.getLocation()[1]

    x = int(x_sum/n)
    y = int(y_sum/n)

    inst.setLocation(x, y)
    inst.setPlacementStatus("PLACED")

    d0 = iterm_by_name(pair[0],"/D").getNet()
    q0 = iterm_by_name(pair[0],"/Q").getNet()
    d1 = iterm_by_name(pair[1],"/D").getNet()
    q1 = iterm_by_name(pair[1],"/Q").getNet()
    clk_net = iterm_by_name(pair[0],"/CLK").getNet()
    reset_net = iterm_by_name(pair[0],"/RESET_B").getNet()
    print(pair[2].getName())
    d2 = iterm_by_name(pair[2],"/D").getNet()
    q2 = iterm_by_name(pair[2],"/Q").getNet()
    d3 = iterm_by_name(pair[3],"/D").getNet()
    q3 = iterm_by_name(pair[3],"/Q").getNet()

    iterm_by_name(inst,"/D0").connect(d0)
    iterm_by_name(inst,"/D1").connect(d1)
    iterm_by_name(inst,"/D2").connect(d2)
    iterm_by_name(inst,"/D3").connect(d3)
    iterm_by_name(inst,"/Q0").connect(q0)
    iterm_by_name(inst,"/Q1").connect(q1)
    iterm_by_name(inst,"/Q2").connect(q2)
    iterm_by_name(inst,"/Q3").connect(q3)
    iterm_by_name(inst,"/CLK").connect(clk_net)
    iterm_by_name(inst,"/RESET").connect(reset_net)

    for i in pair:
        for it in list(i.getITerms()):
            it.disconnect()
        odb.dbInst_destroy(i)
```

Results & Data

Design	TNS	Static Power	Total Power	Wirelength	Area	Total Displacement	Instance Count
gcd_v1	-1.58	0.0	0.00192	16.79	5827.8	1322.01	402
ibex_v1	-18.29	3e-06	0.090743	1129.33	285022.28	66035.61	18509
ibex_v2	-13.0	3e-06	0.09183	1136.18	287801.94	71306.04	18794
jpeg_v1	-0.53	1e-05	0.250611	3782.87	906545.0	162783.37	75519
jpeg_v2	-0.01	1e-05	0.249436	3704.78	904304.22	164380.75	75298
risc32i_v1	-0.17	1e-06	0.01276	560.59	139418.5	24014.35	8901
risc32i_v2	-0.79	1e-06	0.012449	527.95	138520.37	25640.82	8835

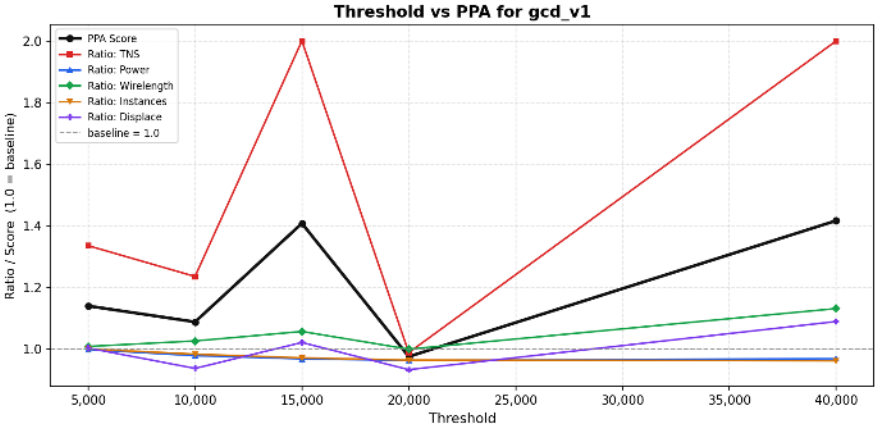
Above is our final PPA figures for each of the 7 designs, using our clustering algorithm. Ultimately, we found that we mostly had similar or better results than what the baseline had for each of the PPA metrics above. Our biggest gains were in performance (total negative slack) in comparison to the baseline, especially for the ibex designs. This was within the realm of our expectations, since we know that the MBFFs will simplify the clock network and open up routing space for the router to better meet timing requirements. Static power and total power consistently was lower for our clustering, which is also expected; the simplified clock network will have reduced clock buffering. The clustering's wirelength, area, and displacement hovered around at around the baseline results, which is good considering that we improved PPA in performance and power. Instance count drops with clustering, which again makes sense since we are grouping instances together.

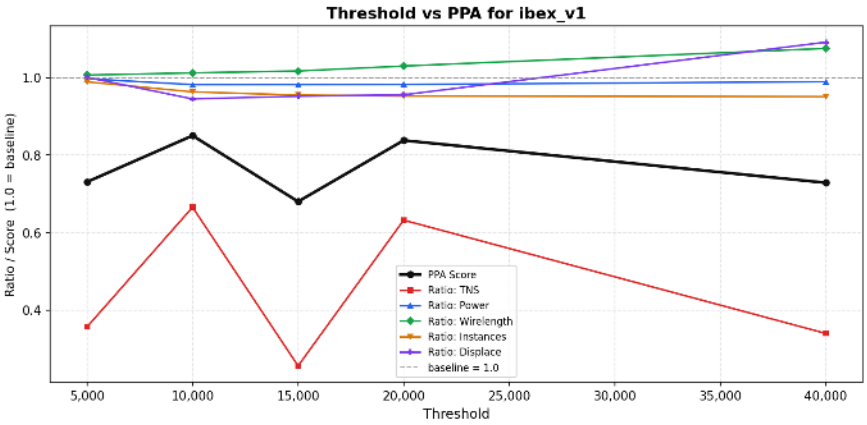
In our approach, recall that we had two thresholds which could be tuned. Firstly, we have the primary distance threshold for initially joining flip flops to make candidate pairs. And secondly, we have the secondary distance threshold for merging the promising clusters of size 2 to make clusters of size 4. Thus, we did a parameter sweep across the search space to give us good QoR.

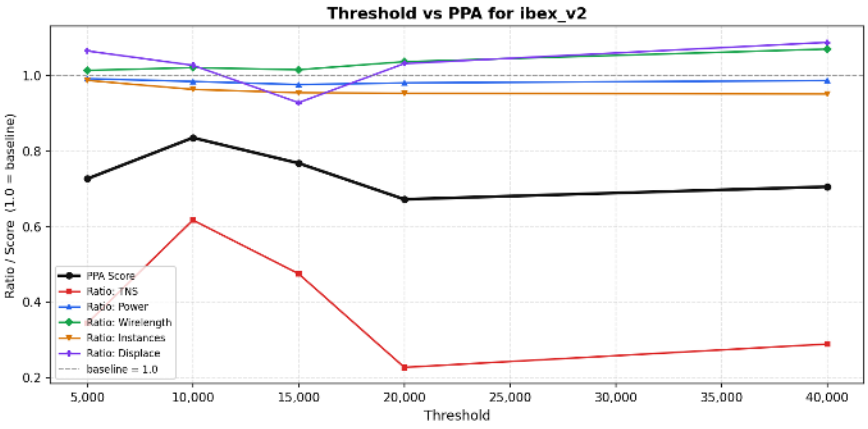
Initially, we swept the primary threshold for making candidate pairs for all the designs and we evaluated the QoR of different thresholds based on PPA metrics such total power, static power,

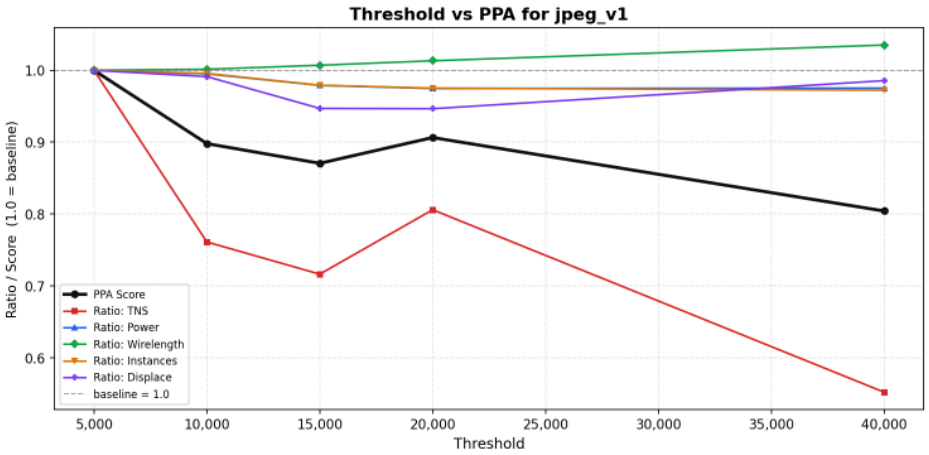
TNS, wirelength, and displacement. The best threshold was selected based on our curated PPA score metric, which is a weighted average of the ratios of the post-clustering PPA metrics over baseline PPA metrics; it takes into account ratios for: total negative slack, total power, total wire length, instance count, total displacement. These different attribute ratios are weighted according to how they are weighted for the competition. In all the graphs below, we plot all the individual ratios as well as the averaged score metric. *The lower the ratio (or the lower the overall score), the better it is.*

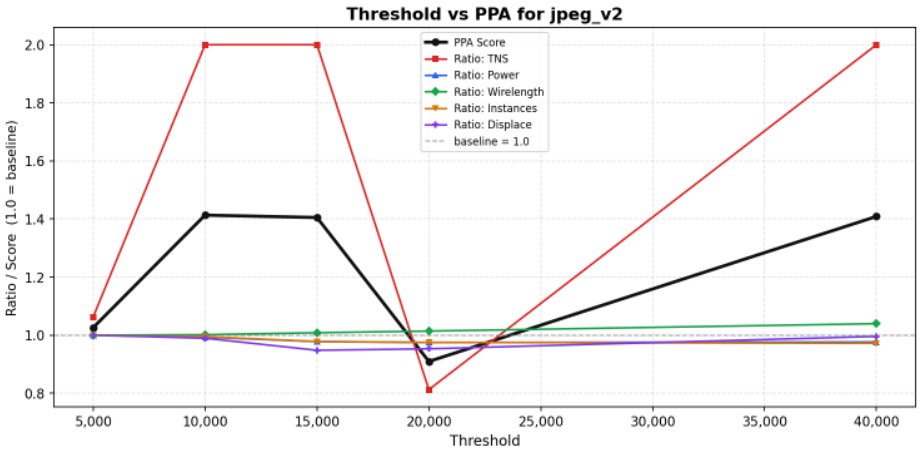
First Threshold (Max distance for a candidate FF pairing)

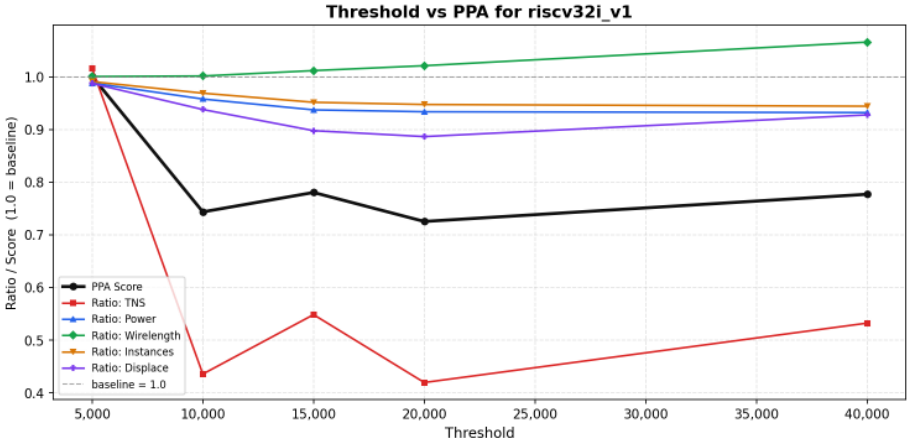


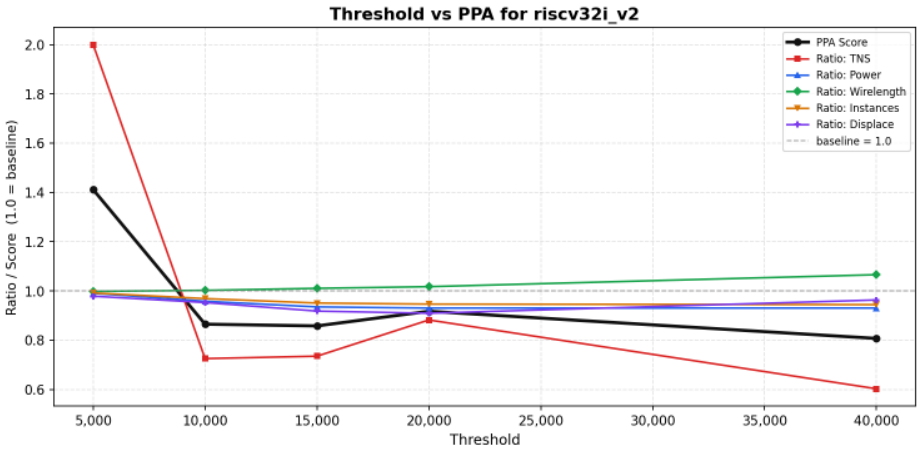





Threshold vs PPA for jpeg_v1
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0

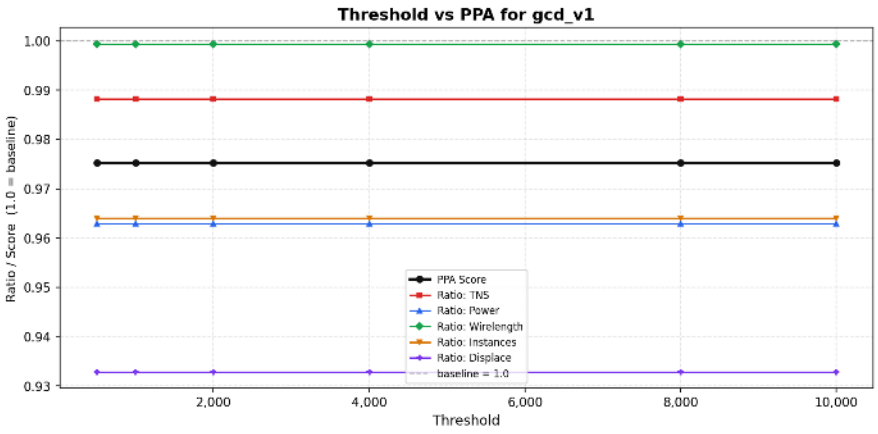

Threshold vs PPA for jpeg_v2
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0

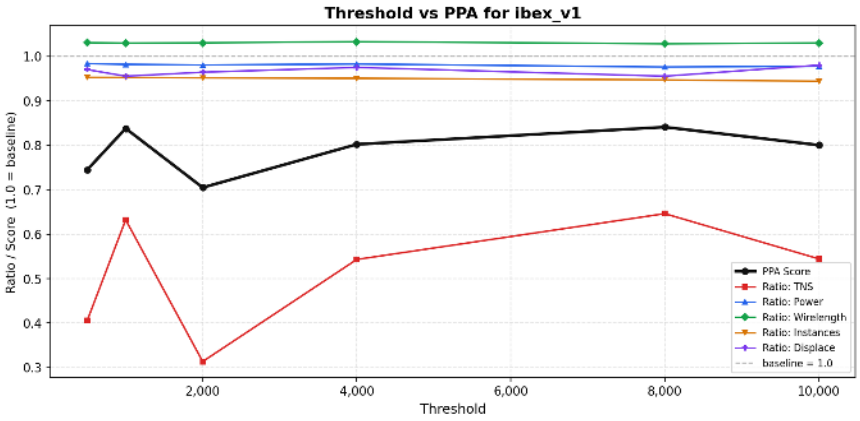

Threshold vs PPA for riscv32i_v1
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0

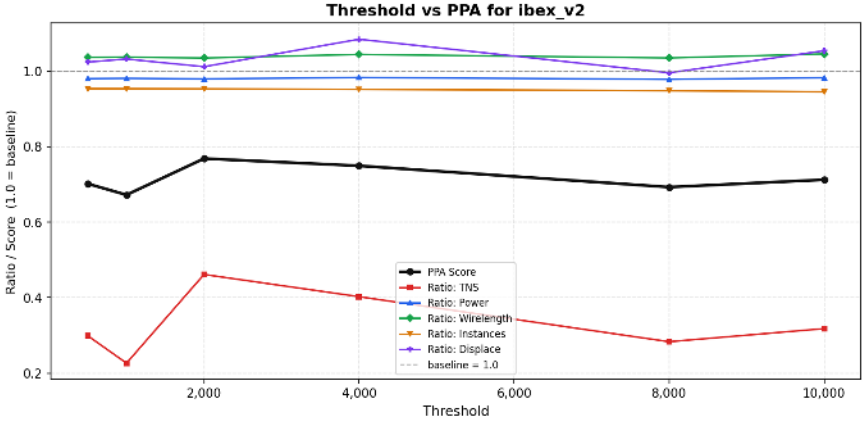


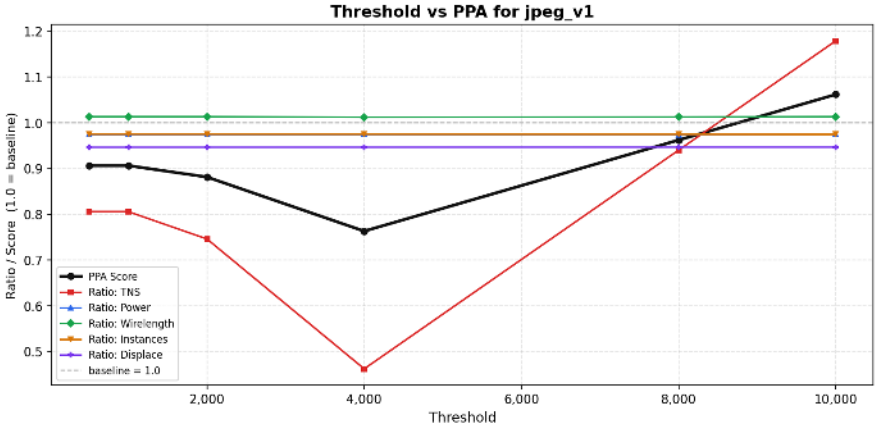
As indicated by a lot of the graphs, a threshold size of 20000 was most ideal in consistently improving PPA for the designs (we verified this by taking an average score across all the designs, and 20000 was best).
Next we fixed the first threshold to this ideal value of 20000 and swept the threshold for the secondary threshold which determines whether clusters of size 2 can be merged into clusters of size 4. The parameter sweep for this threshold was evaluated on the same PPA metrics. The best threshold was finally selected using the same overall score metric as described in the earlier paragraph.

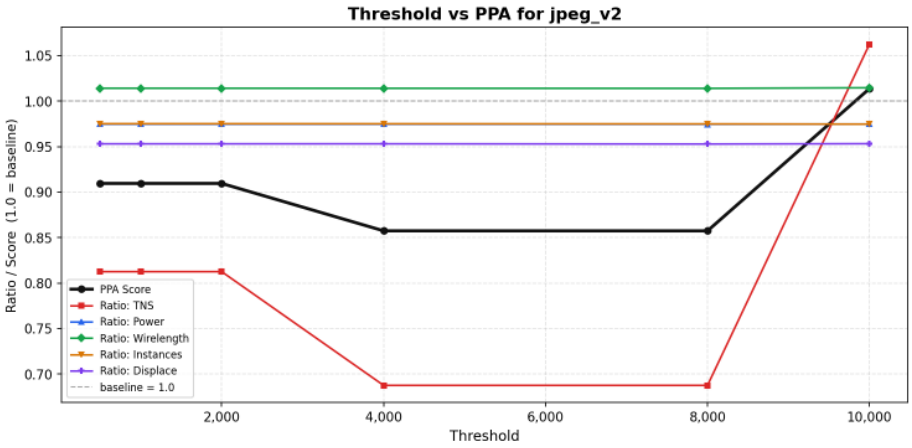
Secondary Threshold (Max distance for a 4-bit MBFF)

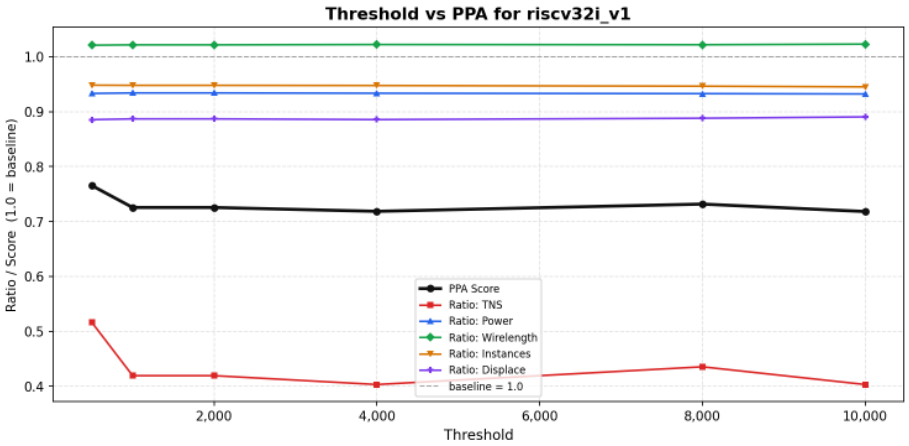


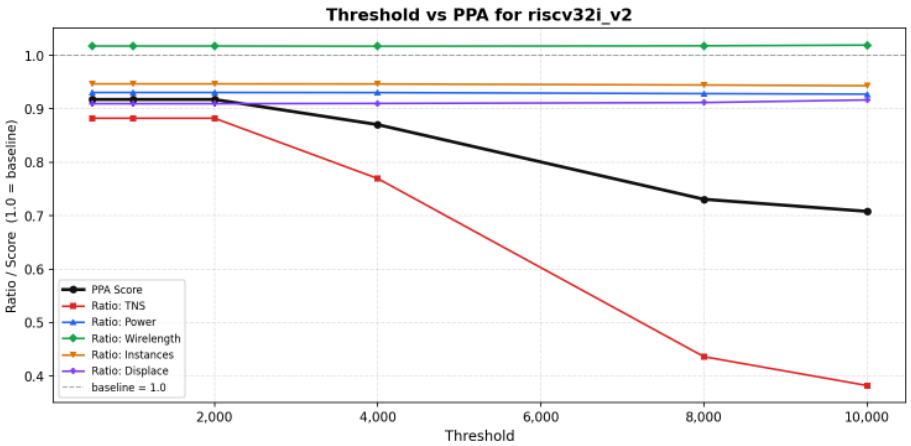

Threshold vs PPA for ibex_v1
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0


Threshold vs PPA for ibex_v2
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0


Threshold vs PPA for jpeg_v1
Ratio / Score (1.0 = baseline)
Threshold
PPA Score
Ratio: TNS
Ratio: Power
Ratio: Wirelength
Ratio: Instances
Ratio: Displace
baseline = 1.0

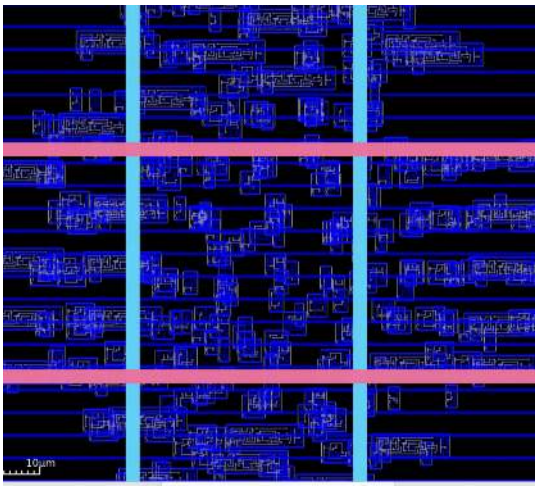




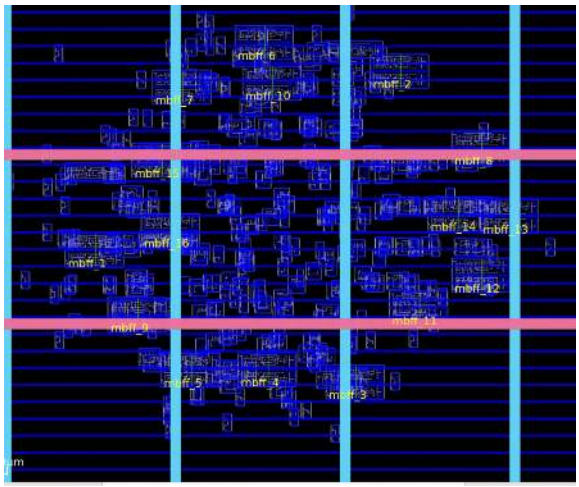


From the score metric averaged across all the designs, the best secondary threshold was 8000. Following this parameter sweep, we set our thresholds for creating candidate pairs of Flip Flops to be 20000 and merging clusters to be 8000.

Closeup Screenshot of GCD_V1 showing MBBFs

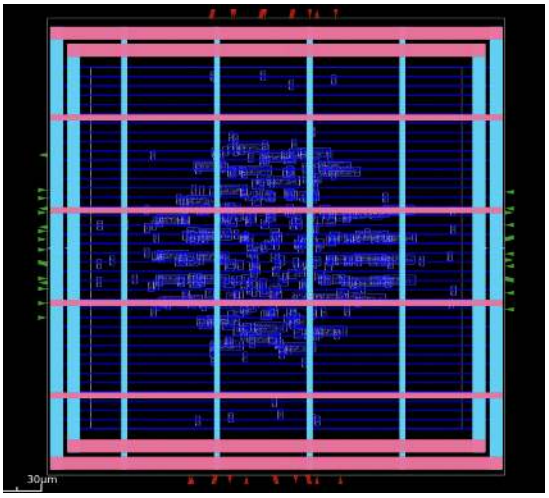


Before

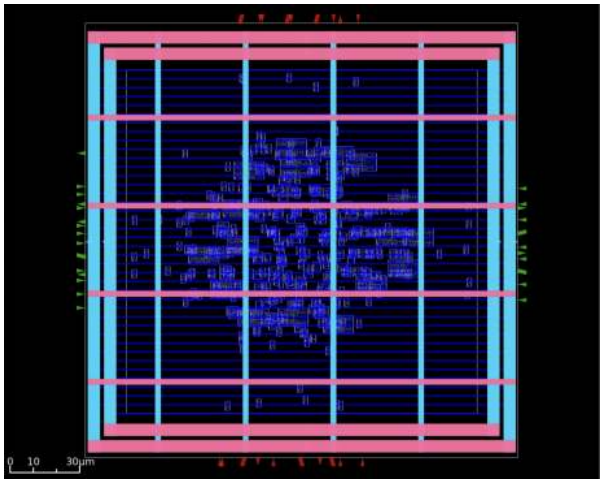


After

Gcd_v1

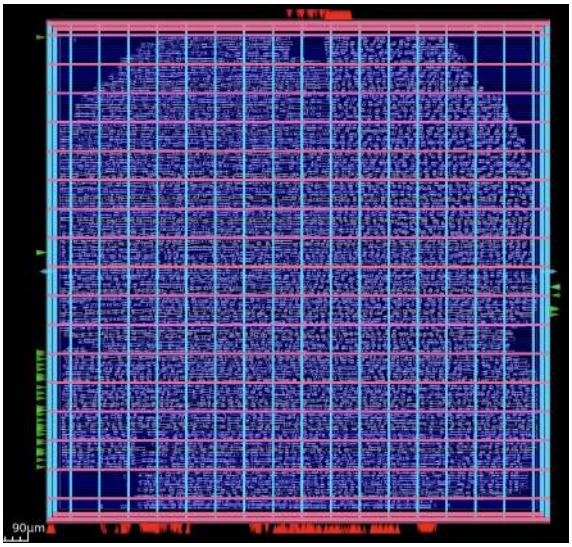
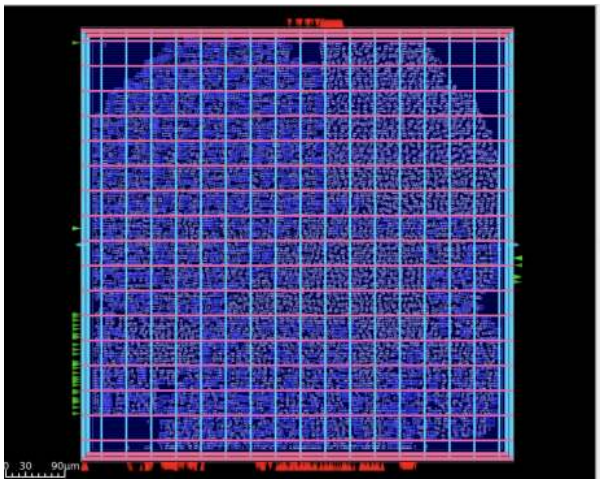


Before

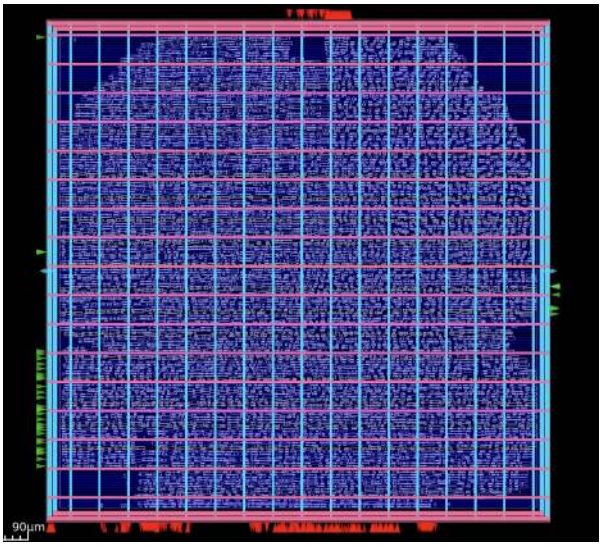
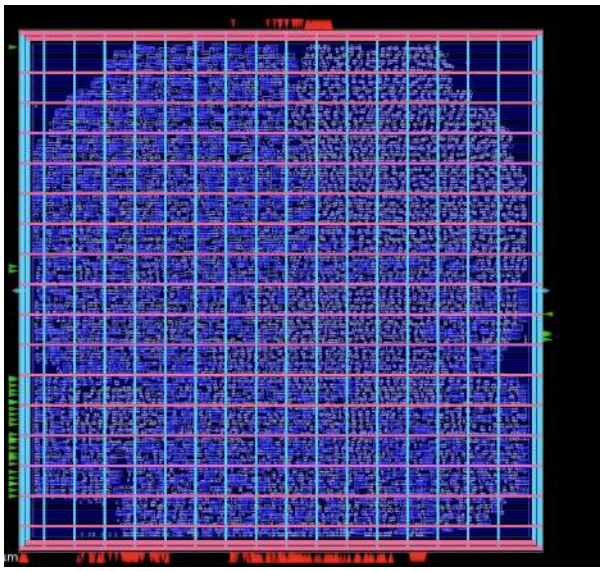


After

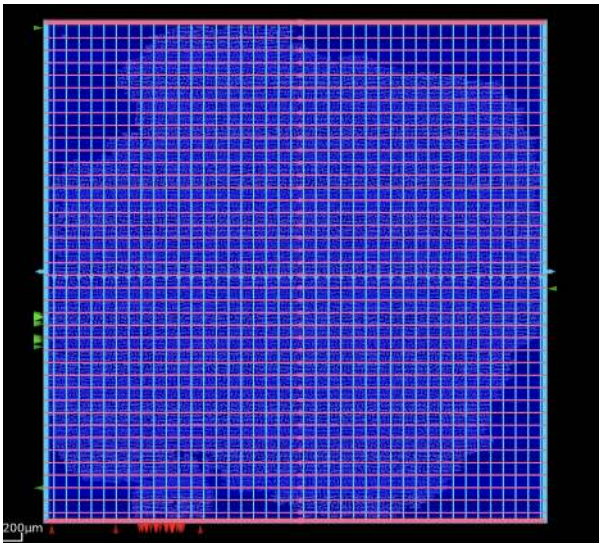
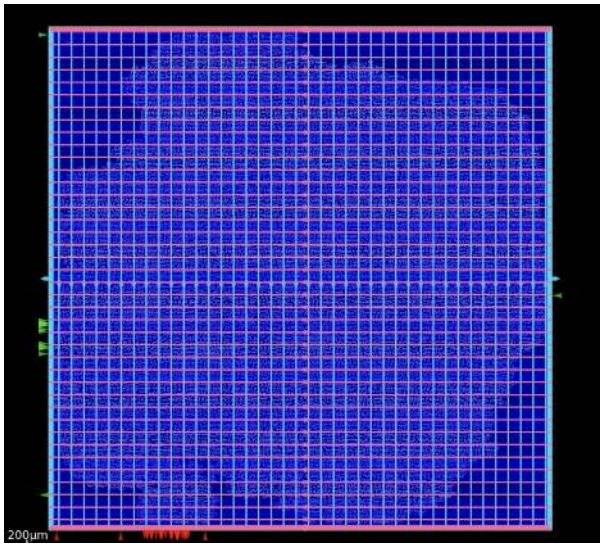
Ibex_v1

 **Before**	 **After**

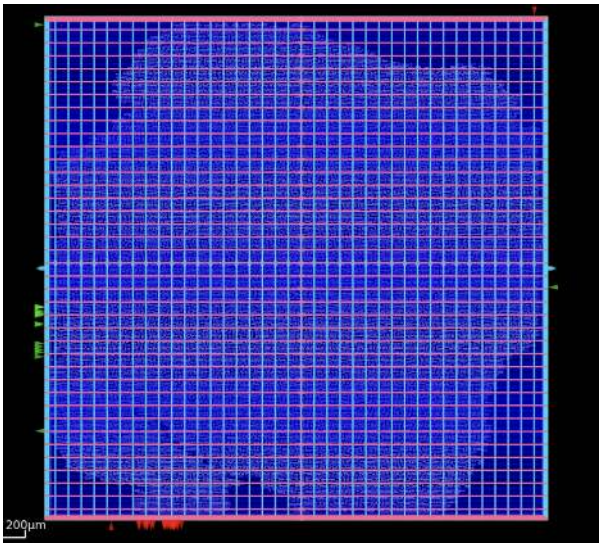
Ibex_v2

 Before	 After

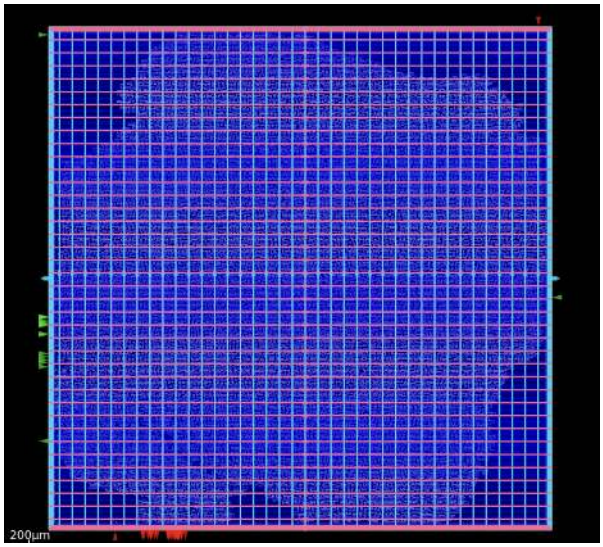
Jpeg_v1

 Before	 After

Jpeg_v2

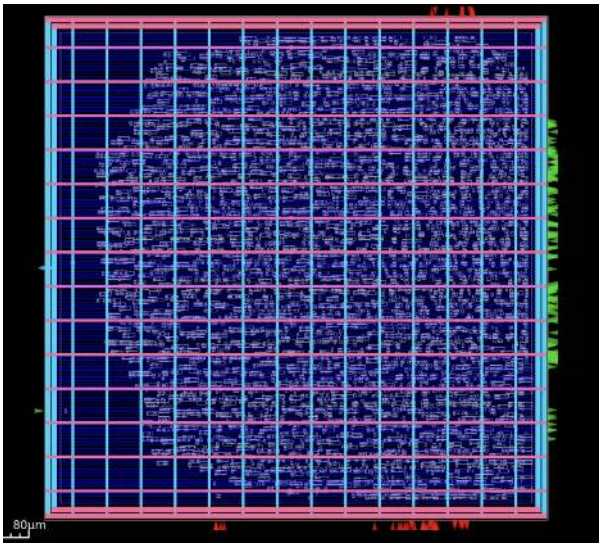


Before

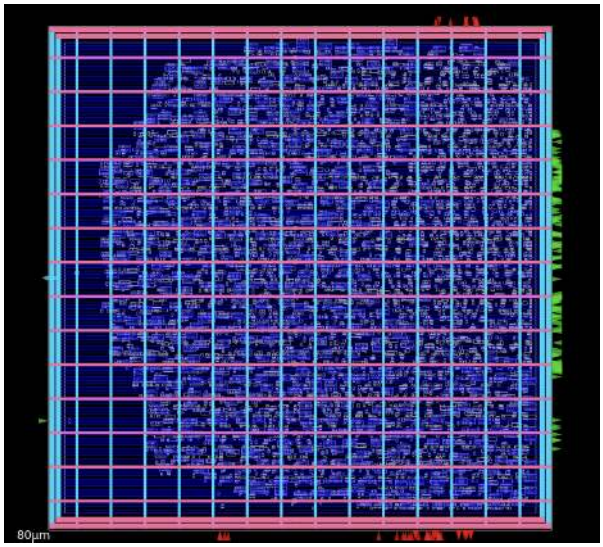


After

Riscv32i_v1

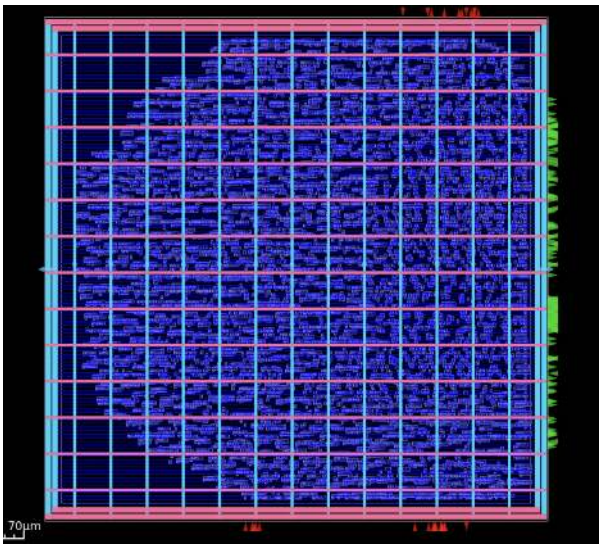
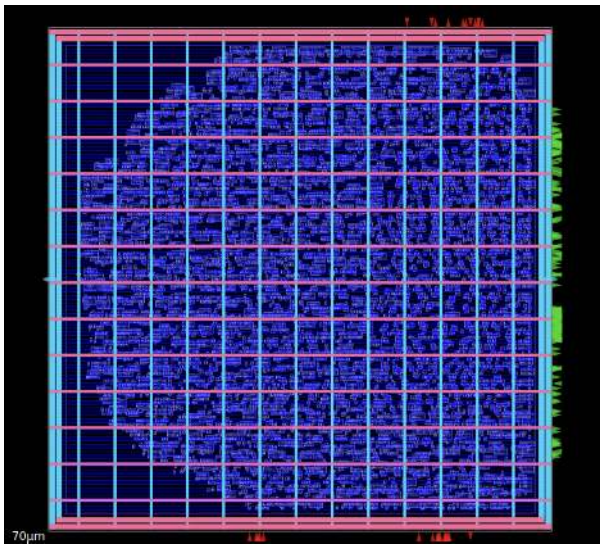


Before



After

Riscv32i_v2

	
Before	After

Validation

What steps did you take to validate the output of your solution? Did you only use evaluator.py? Did you write additional debugging? What were you able to catch and what might still be unverified?
4-10 full sentences. Integrate screenshots, tables, graphs, etc. as needed.

For validation, we mainly took advantage of evaluator.py, as well as equivalence.py (we had to write the changelist json outputting code).

For evaluator.py, we were looking to see if QoR was somewhat better than the baseline, or if there were any errors in the log, and to see if the metrics were sensible. QoR was pretty reasonable and we were satisfied. We also saw reasonable usage of the MBFFs. However, at

the early stages of implementation, we were surprised to find that we got 0 total wirelength for all the designs! Upon closer inspection at the logs, we saw that the MBFFs weren't being placed, and then we realized that we needed to set the placement status for all the new instances we were adding. Now the bug is fixed and wirelengths are sensible.

For equivalence.py, we were looking to see if all checks were passed. At later stages of implementation, we found that ibex designs didn't pass equivalence! Looking at the terminal output, we saw that ibex designs actually had different clock groups, and connecting two FFs from different clock groups was a no-go. This led us to create a compatibility check before declaring two FFs as a candidate pair. Bug fixed, and equivalence passes now.

We also wrote additional debugging through simple print statements throughout the code flow. We printed minimum distances, maximum distances, MBFF instances, power and slack terms, etc. This helped us confirm the implementation flow was working as we were coding it.

We also did validation of QoR for our script's hyperparameters through parameter sweeping, as we showed in the Results section.

Division of Labor

Farhat: I worked initially on exploring different approaches to develop the MBFF autoclustering solution by reviewing papers such as "Two-Stage Multi-bit Flip-Flop Clustering with Useful Skew for Low Power", "GNN-based Multi-bit Flip-flop Clustering and Post-clustering Design Optimization for Energy-efficient 3D", "Revisit MBFF: Efficient Early-Stage Multi-bit Flip-Flops Clustering with Physical and Timing Awareness." After reviewing the literature, I first implemented a naive clustering solution relying solely on physical distance. The method created an adjacency list of neighboring flipflops for each flipflops with flipflops designated as neighbours if the Manhattan distance fell below a threshold. Then, this adjacency list was processed to produce clusters of flip flops. The clusters of flip flops were further processed to produce clean cluster sizes of 4 of and 2 since the MBFFs are only 4 bit or 2 bit. After implementing this approach, I then implemented the timing and power aware approach of "Revisit MBFF: Efficient Early-Stage Multi-bit Flip-Flops Clustering with Physical and Timing Awareness" which is the approach most detailed in the report. Additionally, I performed parameter sweep on the second clustering method I implemented, sweeping the threshold for merging clusters.I also created a script to generate the graphs combing through the results of the parameter sweep. Lastly, I also worked extensively on the write up of the report.

Kevin: I tried out the clustering implementation presented in the paper “Scalable Flip-Flop Clustering Using Divide and Conquer For Capacitated K-Means.” This took several hours, but I wasn’t able to get the implementation to work as well as Farhat’s cluster.py, so we ended up going with his. I did code review for the final cluster.py, debugging errors like the 0 total wirelength error that we mentioned earlier in the report, as well as the ibex equivalence issues also mentioned. I also wrote the parameter sweep script that would perform a semi-grid search for the hyperparameters present in our final clustering algorithm. The sweep took roughly a day. I also helped write up the second half of this report.

We ultimately converged towards the end through sync-up meetings, sharing each others’ code, and writing the report. The main variation from the original plan was when we found tiny bugs in the final clustering algorithm, such as the 0 total wirelength issue, which led us to rerun the hyperparameter sweep again, but it wasn’t too much of a setback. We also found the different-clock-net issue later on (from equivalence.py), but the bug fix only affected the results for the ibex designs, so it also wasn’t too much of a setback.

Conclusion

In our project to develop a MBBF autoclustering engine, we explored multiple different approaches from the available literature. Initially, we implemented a naive clustering algorithm solely based on physical distance between the flip flops. To improve upon this naive approach, we implemented an approach that incorporates a heuristic which considers power and slack and creates a conflict graph of various candidate flip flops. Using this timing and power aware heuristic, the algorithm greedily selects the most optimal pairs for clustering. To optimize the thresholds for this algorithm, (1) threshold for creating candidate pairs of flip flops and (2) threshold for merging clusters of pairs of flip flops, we performed extensive parameter sweeps which evaluated a given threshold value through a score metric that incorporated the different PPA metrics. Overall, through this MBBF clustering approach, clusters of SBBFs were optimally replaced with MBBFs, producing consistent improvements in performance (TNS) and power.

Sources

[1] Andrew B. Kahng, Sayak Kundu, and Shreyas Thumathy. 2024. Scalable Flip-Flop Clustering Using Divide and Conquer For Capacitated K-Means. In Proceedings of the Great Lakes Symposium on VLSI 2024 (GLSVLSI '24). Association for Computing Machinery, New York, NY, USA, 177–184. https://doi.org/10.1145/3649476.3658744

[2] H. -Y. Kao, C. -H. Hsu and S. -H. Huang, "Two-Stage Multi-bit Flip-Flop Clustering with Useful Skew for Low Power," 2019 2nd International Conference on Communication Engineering and Technology (ICCET), Nagoya, Japan, 2019, pp. 178-182, doi: 10.1109/ICCET.2019.8726883.

[3] Yichen Cai, Linyu Zhu, and Xinfei Guo. 2025. Revisit MBFF: Efficient Early-Stage Multi-bit Flip-Flops Clustering with Physical and Timing Awareness. In Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC '25). Association for Computing Machinery, New York, NY, USA, 1230–1236. https://doi.org/10.1145/3658617.3697725